



Swiss Institute of
Bioinformatics

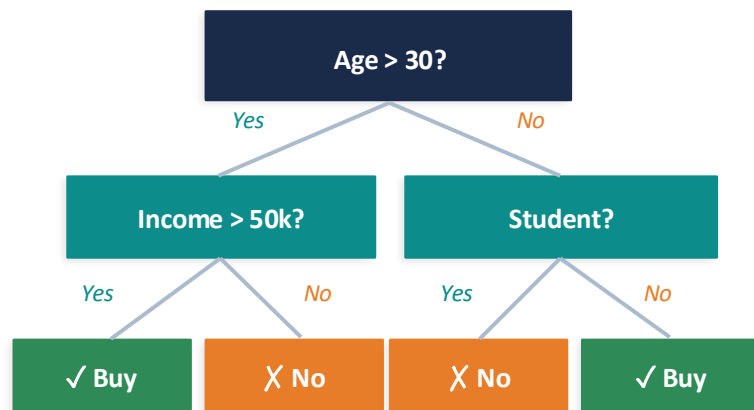
Introduction to Machine Learning techniques

SIB Training Course

22/06/2026

Decision Trees: Learning by asking questions

Game '20 Questions'



Example: Will a customer buy?

Recursive partitioning

At each node, find the feature & threshold that best splits the data. Repeat inside each partition until a stopping rule fires.

Fully interpretable

Able to trace any prediction top-to-bottom. No black box → easy to explain

Handles mixed data natively

Works with continuous and categorical features, missing values, and different scales: no normalisation needed.

Prone to overfitting

A **deep** tree memorises training data perfectly but generalises poorly.

How Trees split: Gini impurity & Information gain

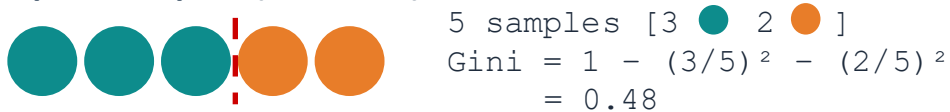
At every node, which split makes the resulting groups most pure?

Gini Impurity

$$Gini = 1 - \sum_k p_k^2$$

p_k = fraction of class k in the node

Before split — impure (Gini ≈ 0.5):



After best split — purer (Gini ≈ 0.0):



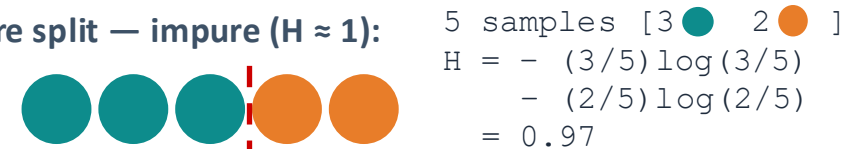
$$Gain = Gini(parent) - \sum \text{weighted } Gini(children)$$

Information Gain (Entropy)

$$H(S) = -\sum_k p_k \log_2(p_k)$$

p_k = fraction of class k in the node

Before split — impure (H ≈ 1):



After best split — purer (H ≈ 0.0):



$$Gain = H(parent) - \sum \text{weighted } H(children)$$

The split threshold is found by sorting each feature and testing every midpoint.

Overfitting & Pruning: Keeping trees honest

$$y = f(x) + \varepsilon$$

Prediction error

$$E \left[\left(y - \hat{f}(x) \right)^2 \right] = E \left[\left(f(x) - \hat{f}(x) \right)^2 \right] + \sigma^2 = \underbrace{\left(f(x) - E \left(\hat{f}(x) \right) \right)^2}_{\text{Bias: systematic error}} + E \left[\underbrace{\left(E \left(\hat{f}(x) \right) - \hat{f}(x) \right)^2}_{\text{Variance: fluctuation}} \right] + \underbrace{\sigma^2}_{\text{Noise}}$$

Too shallow

High bias, low variance

Underfit - missing real patterns in the data.
Training and test error both high.

max_depth = 1–2

Just right

Balanced bias-variance

Generalise well - capturing real structure
without memorising noise.

max_depth = 5–10
(tune with CV)

Too deep

Low bias, high variance

Overfit - memorising training noise.
Low training error, high test error.

max_depth = unlimited

How to regularise a decision tree:

max_depth

Hard ceiling on tree height.

min_samples_split

Do not split a node with fewer than N samples.
Prevents tiny, over-specific splits.

min_samples_leaf

Every leaf must have at least N samples. Smooths
out noisy leaves.

cost-complexity pruning (ccp)

Grow full tree, then remove branches that add
little accuracy vs. complexity.

From one tree to many

Strategy 1: Bagging – bootstrap aggregating

Train independent trees *in parallel* on random subsets.

Deep (low bias, high variance)

Reduces VARIANCE

Less sensitive to outliers

Random Forest

Strategy 2: Boosting

Train trees *sequentially*, each correcting the last mistakes.

Shallow – weak learners (high bias, low variance)

Reduces BIAS

Sensitive to noisy labels (outliers get upweighted).

AdaBoost, Gradient Boosting, LightGBM, XGBoost

Random Forests: Wisdom of the crowd



Random forests := many deliberately different trees => majority vote.

$$\text{Var}(\bar{y}_{RF}) = \rho\sigma^2 + \frac{(1 - \rho)\sigma^2}{T}$$



Sources of randomness:



Bagging (Bootstrap Aggregating)

Each tree trains on a bootstrap sample (~63% of data).
Different trees see different data → make different mistakes.

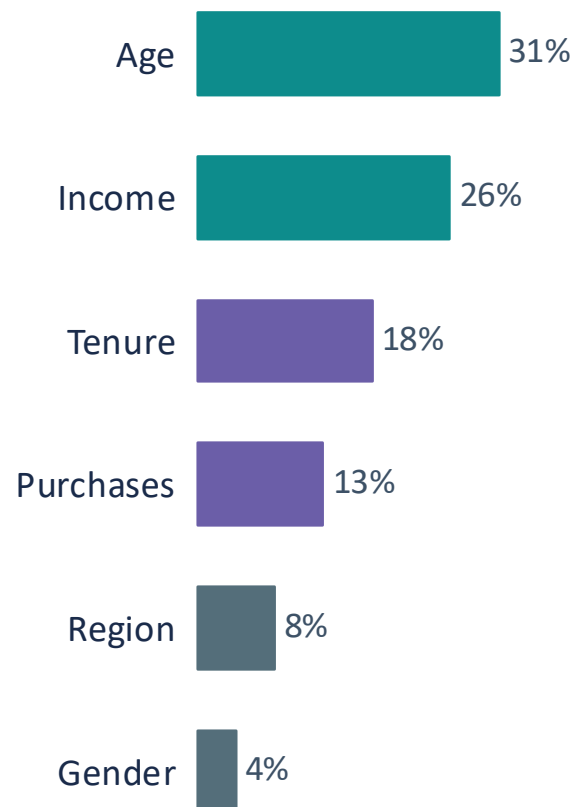


Random feature subsets

At each split, only subset of features are considered (not all). Prevents all trees from using the same dominant feature → true diversity.

Random Forests: Feature importance & Practical tuning

Feature importance (mean Gini decrease)



How much each feature reduces impurity on average across all trees.

Out-of-bag (OOB) error

Each tree trains on ~63% of data

The remaining ~37% = OOB samples -> built-in validation

Key hyperparameters:

n_estimators	More trees = more stable. No overfitting risk from adding trees.
max_features	Lower = more diversity but weaker individual trees.
max_depth	Limiting can speed training with minor accuracy loss.
min_samples_leaf	Ensuring large leaves

AdaBoost: Individually useless, collectively powerful

Sequential training loop

Round 1: uniform weights – every training point matters equally

Train a weak learner (decision stump): $\text{error}_1 < 0.5$

reweight

Round 2: upweight errors – samples wrongly predicted weigh more

Train on reweighted dataset: $\text{error}_2 < 0.25$

reweight

Round 3: repeat

$\text{error}_3 \downarrow \dots$

... (T rounds)

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

Stump weight

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \text{err}_t}{\text{err}_t} \right)$$

High accuracy (low error) $\rightarrow$ large $\alpha \rightarrow$ stump has more say.

Pure chance (error = 0.5) $\rightarrow \alpha = 0 \rightarrow$ stump ignored.

Example (error = 0.25)

$$\alpha = \frac{1}{2} \ln(0.75 / 0.25) = \frac{1}{2} \ln(3) \approx 0.549$$

Weight update per sample i :

$w_i \leftarrow w_i \times \exp(+\alpha)$ if misclassified $\rightarrow \times 1.73$

$w_i \leftarrow w_i \times \exp(-\alpha)$ if correct $\rightarrow \times 0.58$

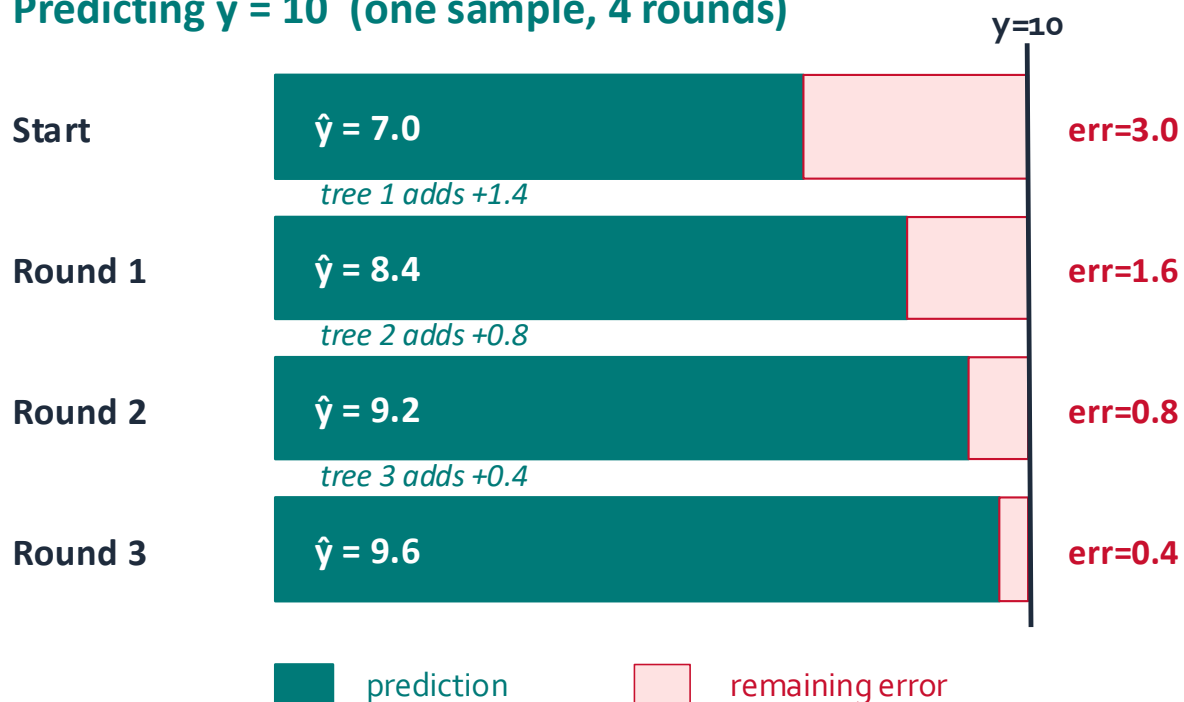
Then renormalise so $\sum w_i = 1$.

Result: next stump must focus on the 1.73× heavier errors.

XGBoost: Closing the error gap

Each new tree looks at the remaining error — the residual — and corrects it.

Predicting $y = 10$ (one sample, 4 rounds)



Four key ideas

1

Residual = what the model still owes

$r_i = y_i - \hat{y}_i$ (MSE) or $-\partial L / \partial \hat{y}_i$ (any loss)
The next tree's training target is this remaining gap.

2

Gradient = direction of improvement

A compass pointing toward lower loss. Each tree is one gradient descent step — but in function space, not parameter space.

3

Hessian = how big a step to take

Curvature of L at $\hat{y}_i$. Flat loss $\rightarrow$ big step. Sharp curve $\rightarrow$ smaller step. Optimal leaf weight: $w^* = -\sum g_i / (\sum h_i + \lambda)$

4

Regularise: don't chase every residual

$\Omega(f) = \gamma \cdot (\text{leaf count}) + \frac{1}{2}\lambda \|\text{leaf weights}\|^2$
Prevents trees from memorising noise in small residuals.

AdaBoost: reweight samples $\rightarrow$ Gradient Boosting: fit residuals $\rightarrow$ XGBoost: + regularise + 2nd-order Taylor

Classification asks ***which class***. Regression asks ***how much***.
The models are the same, only the loss is different.

RF for Regression: What matters

CLASSIFICATION

Discrete output

Vote

Split: Gini impurity, Entropy reduction

Example: Is this fruit an orange or an apple?

REGRESSION

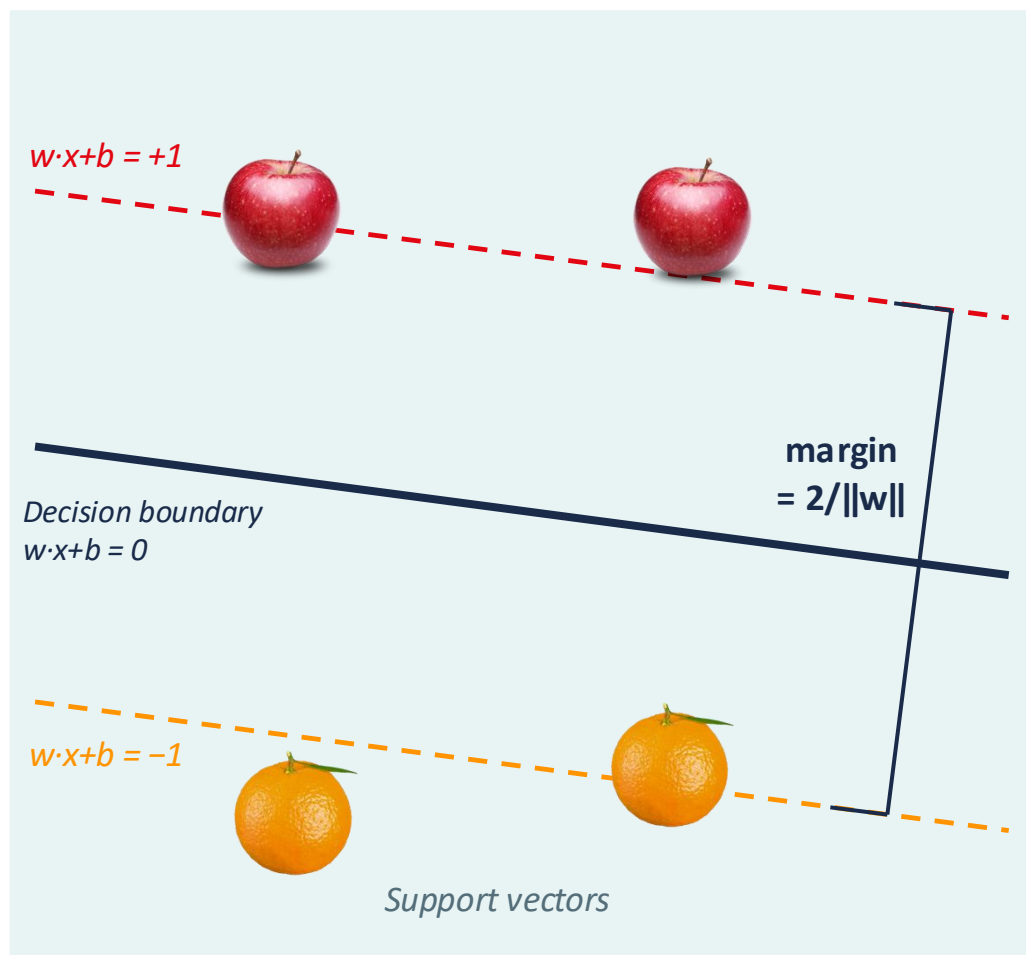
Continuous output

Average

Split: variance reduction, MSE reduction

Example: How much does this fruit cost?

The margin: Measuring the breathing room



What is the margin?

- The margin := the gap between the two classes around the decision boundary
- $2/\|w\|$ inversely proportional to the weight vector length
- Wider margin → more confident classification

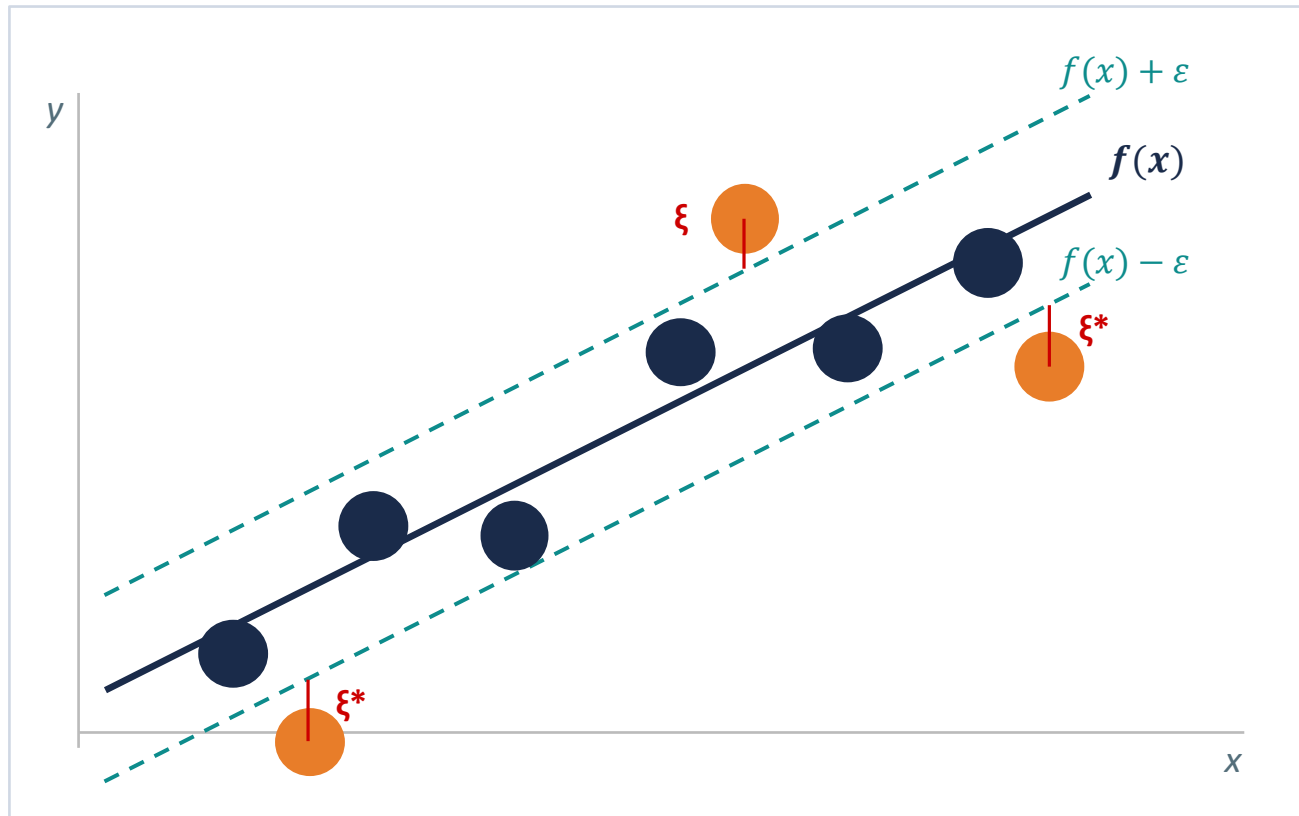
What are support vectors?

- Training points touching the dashed lines → 'support' the boundary position
- Remove a non-support vector → boundary unchanged
- Remove a support vector → boundary shifts

SVM for Regression: Flipping the idea

Classification: keep points outside the margin tube

Regression: keep points inside a tube around the prediction



• inside tube = no penalty • outside = penalty ξ^*

ϵ -insensitive loss

$$L(y, f(x)) = \max(0, |y - f(x)| - \epsilon)$$

Zero loss inside the tube. Linear penalty outside.

SVR objective

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i (\xi_i + \xi_i^*)$$

$\xi_i \geq 0$: above tube, $\xi_i^* \geq 0$: below tube

ϵ Tube width:

Controls how much error is free. Larger $\epsilon \rightarrow$ simpler model, more bias.

C

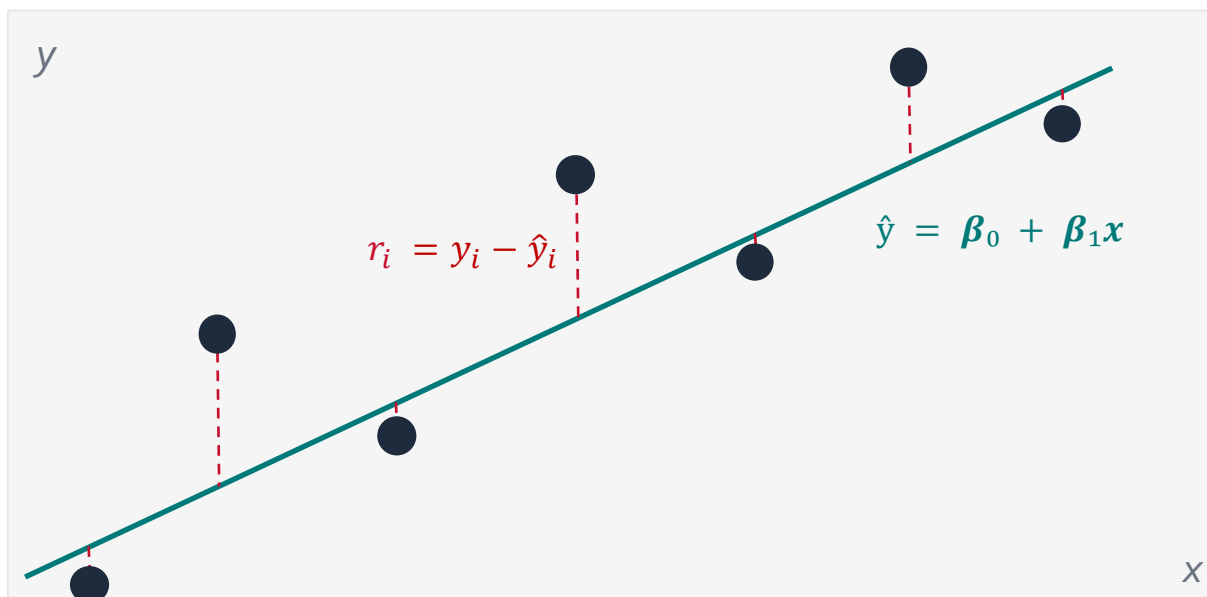
Penalty weight:

Large C punishes violations

Ordinary Least Squares

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

What OLS does visually



$$\text{minimise } RSS = \sum r_i^2 = \sum (y_i - \hat{y}_i)^2$$

The closed-form solution

$$\text{Set } \partial RSS / \partial \beta = 0 \rightarrow X'X\beta = X'y$$

$$\hat{\beta} = (X'X)^{-1}X'y \text{ (exact, no iterations)}$$

Coefficient β_j

Change in $\hat{y}$ per 1-unit increase in x , all others held fixed.

$R^2 = 1 - RSS/TSS$

Fraction of variance explained.

Multicollinearity

Correlated features $\rightarrow X'X$ near-singular $\rightarrow \hat{\beta}$ blows up. Fix: Ridge

$p \gg n$

$X'X$ rank-deficient $\rightarrow$ no unique solution. Fix: Lasso or Elastic Net

When OLS fails: ill-conditioning and high dimensions

Adding a penalty to the loss function shrinks coefficients: trading a small bias increase for a large variance reduction.

$$\text{Penalised loss} = \text{RSS} + \lambda \times \text{Penalty}(\beta) \qquad \lambda = 0 : \text{OLS} \qquad \lambda \rightarrow \infty : \beta = 0$$

OLS instability

$$\hat{\beta} = (X'X)^{-1}X'y$$

Almost correlated columns $\rightarrow X'X$ near-singular
 $\|\hat{\beta}\| \rightarrow \infty$, unstable predictions

When does OLS fail?

Always standardise features before regularizing.

$p \gg n$

$X'X$ is rank-deficient (singular). No unique OLS solution.
 Lasso/Elastic Net.

Multicollinearity

Correlated features $\rightarrow X'X$ ill-conditioned $\rightarrow \|\hat{\beta}\|$ large & unstable. Ridge.

Feature selection

Want model to automatically zero-out irrelevant variables.
 Lasso (L1 penalty).

High-dimensional bias

Adding many features always improves training R^2 . Penalising β controls generalisation.

Ridge, Lasso, Elastic Net: constraint regions & sparsity

Ridge (L2)

$$\text{Penalty} = \sum \beta_j^2$$

Shrinks all coefficients toward zero.
Rarely to exactly zero: no feature selection.

Best when: Correlated features; want stable, dense coefficients; $p > n$

Lasso (L1)

$$\text{Penalty} = \sum |\beta_j|$$

Shrinks some coefficients to EXACTLY zero: automatic feature selection, sparse model.

Best when: Many features, few truly matter; interpretability + selection needed

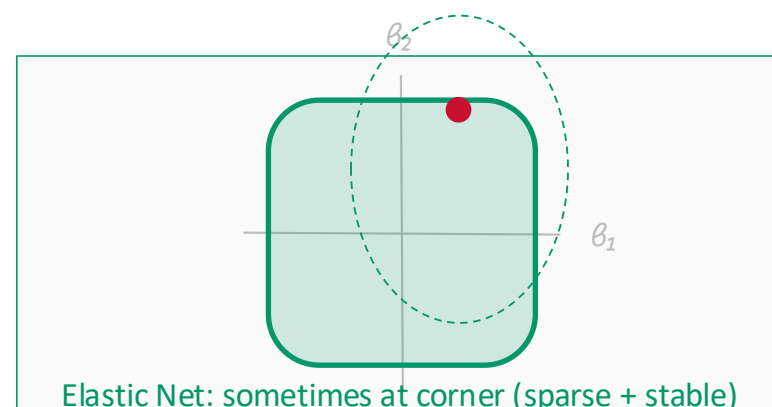
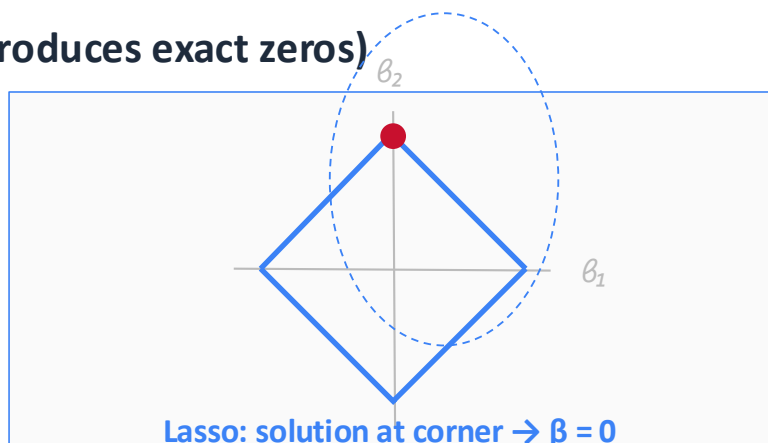
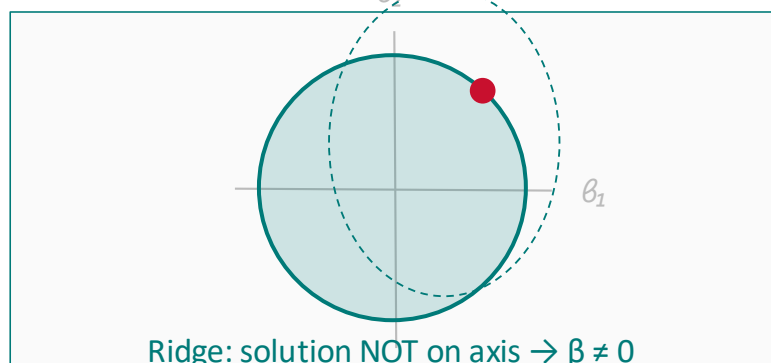
Elastic Net

$$\text{Penalty} = l_1 \sum |\beta_j| + l_2 \sum \beta_j^2$$

L1 sparsity + L2 stability: selects features while handling correlated groups.

Best when: Correlated features + sparsity required. Safest default in high dimensions

Constraint region geometry (why Lasso produces exact zeros)



Regression model evaluation

Mean Squared Error MSE $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$

Same scale as y^2 . Sensitive to outliers.

Root Mean Squared Error RMSE $\sqrt{MSE}$

Bring the error back to the same unit as y .

Mean Absolute Error $\frac{1}{n} \sum |y_i - \hat{y}_i|$

More robust to outliers than RMSE. Sensitive to proportionate errors.

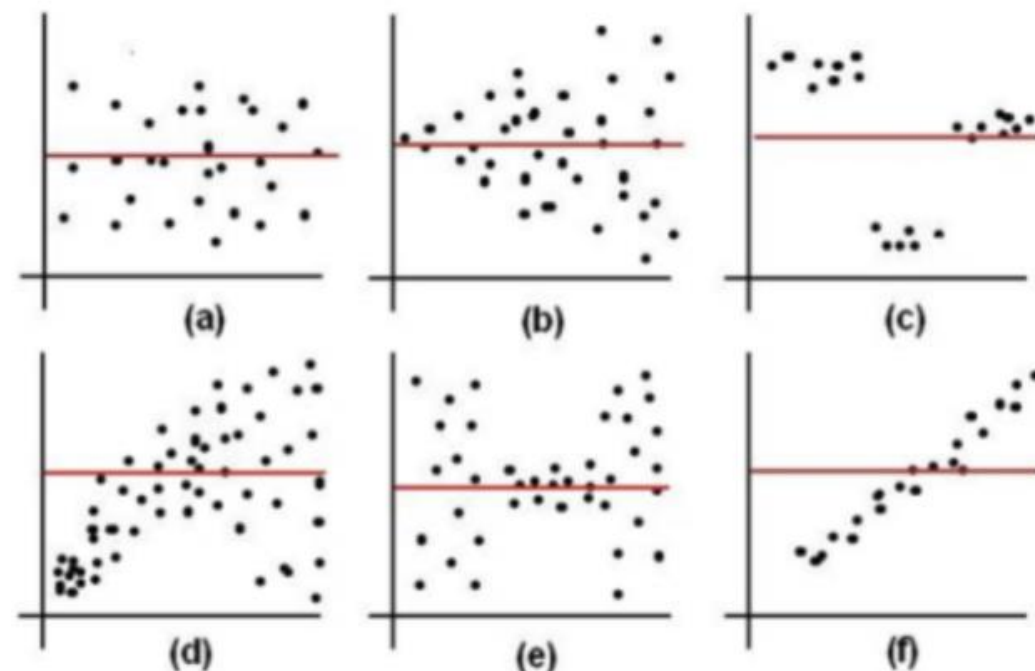
Coefficient of determination R^2 $1 - RSS/TSS$

Fraction of variance in y explained by the model. 1, perfect. 0, like predicting average for all. < 0 , even worse. Reward overfitting.

Adjusted R^2 $1 - \frac{RSS}{n-p-1} / \frac{TSS}{n-1}$

Measure goodness of fit while penalizing unnecessary complexity. Avoid overfitting.

Cross-validation



Residual plot